

EXPERIMENT – 1

Title

Installation of Hadoop on Windows Operating System

Aim

To install and verify Apache Hadoop (version 3.3.6) on a Windows system.

Software Requirements

- Operating System : Windows 10 / 11 (64-bit)
- Java Version : JDK 1.8
- Hadoop Version : Apache Hadoop 3.3.6
- Tool : Command Prompt

Procedure

1. Download **Java JDK 1.8** and install it in C:\Java.
2. Set **JAVA_HOME** environment variable:
 - Variable name: JAVA_HOME
 - Variable value: C:\Java
3. Add Java and Hadoop paths to **System PATH**:
 - C:\Java\bin
 - C:\hadoop-3.3.6\bin
 - C:\hadoop-3.3.6\sbin
4. Download **Apache Hadoop 3.3.6** and extract it to C:\hadoop-3.3.6.
5. Copy winutils.exe to C:\hadoop-3.3.6\bin.
6. Configure hadoop-env.cmd file:
 - Path: C:\hadoop-3.3.6\etc\hadoop\hadoop-env.cmd
 - Set JAVA_HOME path correctly.

7. Open **Command Prompt as Administrator**.
8. Verify Java installation using:
 - java -version
9. Verify Hadoop installation using:
 - hadoop version
10. Format the Hadoop NameNode:
 - hdfs namenode -format
11. Start Hadoop services:
 - start-dfs.cmd

Output

The Hadoop version and NameNode service were successfully displayed without any errors.

Result

Apache Hadoop was successfully installed and verified on the Windows system.

EXPERIMENT – 2

Title

Configuration of Hadoop and Performing Basic HDFS Operations

Aim

To configure Hadoop in pseudo-distributed mode and perform basic HDFS file system operations.

Procedure

Step 1: Hadoop Configuration

1. Configure core-site.xml:
 - Location: C:\hadoop-3.3.6\etc\hadoop\

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```
2. Configure hdfs-site.xml:
 - ```
<configuration>
```
  - ```
<property>
```
 - ```
<name>dfs.replication</name>
```
  - ```
<value>1</value>
```
 - ```
</property>
```
  - ```
</configuration>
```
3. Start Hadoop services:

```
start-dfs.cmd
```
4. Verify running services:

```
jps
```

Step 2: Basic HDFS Operations

1. Create a directory in HDFS:

```
hdfs dfs -mkdir /input
```

2. List directories:

```
hdfs dfs -ls /
```

3. Create a text file in local system and upload it to HDFS:

```
hdfs dfs -put input.txt /input
```

4. View contents of HDFS file:

```
hdfs dfs -cat /input/input.txt
```

5. Copy file from HDFS to local system:

```
hdfs dfs -get /input/input.txt
```

6. Delete file from HDFS:

```
hdfs dfs -rm /input/input.txt
```

Output

The input file was successfully stored, accessed, and removed from the Hadoop Distributed File System (HDFS).

Result

Hadoop was successfully configured and basic HDFS operations were performed.

EXPERIMENT – 3

Title

Word Count of a Text File using Hadoop Java JAR

Aim

To count the number of occurrences of each word in a text file using Hadoop MapReduce Java JAR.

Software Requirements

- Windows OS
- Java JDK 1.8
- Apache Hadoop 3.3.6

Procedure

1. Start Hadoop services:

```
start-dfs.cmd
```

2. Create an input directory in HDFS:

```
hdfs dfs -mkdir /input
```

3. Create a sample text file (`input.txt`) in the local system.

4. Upload the file to HDFS:

```
hdfs dfs -put input.txt /input
```

5. Run the Hadoop WordCount example JAR:

```
hadoop jar %HADOOP_HOME%\share\hadoop\mapreduce\hadoop-mapreduce-examples-3.3.6.jar wordcount /input /output
```

6. View the output:

```
hdfs dfs -cat /output/part-r-00000
```

Output

Each word and its corresponding count are displayed on the console.

Result

The word count operation was successfully performed using Hadoop MapReduce Java JAR.

EXPERIMENT – 4

Title

Word Count of a Text File using Hadoop Streaming with Python

Aim

To count the number of occurrences of each word in a text file using **Hadoop Streaming and Python Mapper & Reducer programs**.

Software Requirements

1. Windows OS
2. Java JDK 1.8
3. Apache Hadoop 3.3.6
4. Python 3.x

Procedure

Step 1: Create Input Directory in HDFS

```
hdfs dfs -mkdir /input
```

Step 2: Upload Text File to HDFS

```
hdfs dfs -put input.txt /input
```

Step 3: Create Python Mapper (mapper.py)

```
1. #!/usr/bin/env python
2. import sys
3.
4. for line in sys.stdin:
5.     line = line.strip()
6.     words = line.split()
7.     for word in words:
8.         print(f"{word}\t1")
```

Step 4: Create Python Reducer (reducer.py)

```

1. #!/usr/bin/env python
2. import sys
3.
4. current_word = None
5. current_count = 0
6.
7. for line in sys.stdin:
8.     word, count = line.strip().split("\t")
9.     count = int(count)
10.
11.     if current_word == word:
12.         current_count += count
13.     else:
14.         if current_word:
15.             print(f"{current_word}\t{current_count}")
16.             current_word = word
17.             current_count = count
18.
19. if current_word:
20.     print(f"{current_word}\t{current_count}")

```

Step 5: Run Hadoop Streaming Job

```

1. hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-
   streaming-3.3.6.jar ^
2. -input /input ^
3. -output /output_py ^
4. -mapper "python mapper.py" ^
5. -reducer "python reducer.py"

```

Step 6: View Output

```
hdfs dfs -cat /output_py/part-00000
```

Output

Each word along with its frequency is displayed.

Result

The word count operation was successfully performed using **Hadoop Streaming with Python**.